Q : 1 Introduction to Python and its Features (simple, high-level, interpreted language).

Ans : Python is a high-level, interpreted programming language that is widely used in software development, web development, data science, artificial intelligence, and automation. It was created by Guido van Rossum and first released in 1991. Python is known for its simple and readable syntax, which makes it easy to learn and use. Because of its simplicity and powerful features, Python has become one of the most popular programming languages in the world.

Simple and Easy to Learn:
Python has a simple and clear syntax that is like the English language. This makes it easy for beginners to understand and learn programming concepts.

High-Level Language:
Python is a high-level language, which means programmers do not need to manage memory or hardware directly. The language handles many complex tasks automatically.

Interpreted Language:
Python is an interpreted language, meaning the code is executed line by line by the Python interpreter. This makes debugging easier and faster.

Q : 2 History and evolution of Python.

Ans : Python 1.x (1991–2000):
Python 1.0 was released in 1994. It introduced many important features such as functional programming tools including lambda, map, filter, and reduce. During this time, Python started gaining popularity among developers because of its simplicity and flexibility.

Python 2.x (2000–2010):
Python 2.0 was released in 2000 and introduced new features such as list comprehensions, garbage collection, and support for Unicode. Python 2 became widely used in many industries and remained popular for many years. However, due to some design limitations, developers later decided to improve the language further with a new major version.

Python 3.x (2008–Present):
Python 3.0 was released in 2008 as an improved and more consistent version of the language. Some older features were removed or modified to make the language cleaner and more efficient. Python 3 introduced better support for Unicode, improved libraries, and better performance.

Today, Python 3 is the most widely used version of Python, and Python 2 reached its end of life in January 2020, meaning it is no longer supported or updated.

Q : 3 Advantages of using Python over other programming languages.

Ans :

- Easy to Learn and Read
- High-Level Language

- Large Standard Library
- Platform Independent
- Strong Community Support
- Supports Multiple Programming Paradigms

Q : 4 Writing and executing your first Python program.

Ans :
```
print("Hello, World!")
```

Q : 5 Understanding Python's PEP 8 guidelines.

Ans :

- Indentation:
  Python uses indentation to define code blocks. PEP 8 recommends using 4 spaces per indentation level.
- Use of Blank Lines:
  Blank lines should be used to separate different sections of code, such as functions and classes, to make the code more readable.
- Import Statements:
  Imports should be written at the top of the file and usually on separate lines.
- Spaces Around Operators:
  Spaces should be used around operators and after commas for better readability.
- Comments and Documentation:
  Comments should clearly explain the purpose of the code. Good comments improve code understanding.

Q : 6 Indentation, comments, and naming conventions in Python.

Ans :

- Indentation in Python :

  Indentation refers to the spaces used at the beginning of a line to define a block of code.

  Key Points:

  - Python uses indentation instead of braces {} to define code blocks.
  - The standard indentation is 4 spaces per level.
  - Proper indentation is necessary; otherwise, it will result in an error.
- Comments in Python :
  Comments are used to explain the code. They are ignored by the Python interpreter and are only meant for human understanding.
- Naming Convention in Python :
  Naming conventions are rules for naming variables, functions, classes, and constants to make code more readable and consistent.

Q : 7 Writing readable and maintainable code.

Ans :

- Use Meaningful Names.
  Choose clear and descriptive names for variables, functions, and classes.
  s
- Follow Proper Indentation and Formatting

  Use consistent indentation (4 spaces) and proper spacing to make code structure clear.

- Write Simple and Clear Code :
  Avoid complex logic. Break large problems into smaller parts.

- Use Comments and Documentation
  Write comments to explain complex logic or important steps.

Q : 8 Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets.

Ans : In Python, a data type defines the type of value a variable can store. Understanding data types is important because it helps in performing operations correctly and managing data efficiently. Python provides several built-in data types such as integers, floats, strings, lists, tuples, dictionaries, and sets.

1. Integer (int) :

An integer is a whole number without any decimal point. It can be positive, negative, or zero.

2. Float (float) :

A float is a number that contains a decimal point.

3. String (str) :

A string is a sequence of characters enclosed in single quotes (' ') or double quotes (" ").

4. List (list) :

A list is an ordered collection of items. It is mutable, meaning its elements can be changed.

5. Tuple (tuple) :

A tuple is like a list but is immutable, meaning its elements cannot be changed after creation.

6. Dictionary (dict) :

A dictionary is a collection of key-value pairs. Each key is unique and is used to access its corresponding value.

7. Set (set)

A set is an unordered collection of unique elements. It does not allow duplicate values.

Q : 9 Python variables and memory allocation.

Ans : In Python, variables are used to store data values. Unlike some other programming languages, Python does not require explicit declaration of variables. A variable is created automatically when a value is assigned to it. Python also manages memory automatically, which makes it easier for programmers to work without worrying about low-level memory details.

1. Variables in Python

A variable is a name that refers to a value stored in memory. Python is a dynamically typed language, which means the type of a variable is determined at runtime.

2. Memory Allocation in Python

Memory allocation is the process of assigning memory space to store variables and their values.

In Python, memory is managed automatically by the Python interpreter.

Q : 10 Python operators: arithmetic, comparison, logical, bitwise.

Ans :

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical operations such as addition, subtraction, multiplication, and division.

```
a = 10

b = 3

print(a + b)   # Addition → 13

print(a - b)   # Subtraction → 7

print(a * b)   # Multiplication → 30

print(a / b)   # Division → 3.33

print(a // b)  # Floor Division → 3

print(a % b)   # Modulus → 1

print(a ** b)  # Exponentiation → 1000
```

2. Comparison Operators

Comparison operators are used to compare two values and return a boolean result (True or False).

```
a = 10

b = 5

print(a == b)  # Equal to → False

print(a != b)  # Not equal → True

print(a > b)   # Greater than → True
```

```
print(a < b)   # Less than → False
print(a >= b)  # Greater than or equal → True
print(a <= b)  # Less than or equal → False
```

3. Logical Operators

Logical operators are used to combine conditional statements.

```
x = True
y = False
print(x and y)  # Logical AND → False
print(x or y)   # Logical OR → True
print(not x)    # Logical NOT → False
```

4. Bitwise Operators

Bitwise operators perform operations on binary (bit-level) values of integers.

```
a = 5   # Binary: 0101
b = 3   # Binary: 0011
print(a & b)   # AND → 1
print(a | b)   # OR → 7
print(a ^ b)   # XOR → 6
print(~a)      # NOT → -6
print(a << 1)  # Left shift → 10
print(a >> 1)  # Right shift → 2
```

Q : 11 Introduction to conditional statements: **`if, else, elif`**.

Ans :

1. The if Statement

   The if statement is used to execute a block of code only when a condition is True.

```
age = 18

if age >= 18:
    print("You are eligible to vote")
```

2. The else Statement

   The else statement is used to execute a block of code when the condition in the if statement is False.

```
age = 16
```

```
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

3. The elif Statement

The elif (else if) statement is used to check multiple conditions. It is executed when the previous conditions are false.

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Q : 12 Nested if-else conditions.

Ans :

```
num = 10
if num > 0:
    if num % 2 == 0:
        print("Positive even number")
    else:
        print("Positive odd number")
else:
    print("Number is not positive")
```

Q : 13 Introduction to for and while loops.

Ans :

1. The for Loop

A for loop is used to iterate over a sequence such as a list, tuple, string, or range of numbers.

```
for i in range(1, 6):
    print(i)
```

2. The while Loop

A while loop executes a block of code as long as a given condition is True.

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

Q : 14 How loops work in Python.

Ans : Loops in Python are used to execute a block of code repeatedly. They help automate repetitive tasks and make programs more efficient. Python mainly provides two types of loops: for loop and while loop.

1. Initialize (starting point)
2. Check condition
3. Execute code block
4. Update value
5. Repeat until condition becomes false

Q : 15 Using loops with collections (lists, tuples, etc.).

Ans : In Python, collections such as lists, tuples, strings, dictionaries, and sets store multiple values. Loops are commonly used to iterate (traverse) through these collections and perform operations on each element. This makes it easier to process large amounts of data efficiently.

1. Looping Through a List :

```
numbers = [1, 2, 3, 4]

for num in numbers:
    print(num)
```

2. Looping Through a Tuple :

```
data = (10, 20, 30)

for item in data:
    print(item)
```

3. Looping Through a String :

```
name = "Python"
```

```
for char in name:
    print(char)
```

Q : 16 Built-in methods for strings, lists, etc.

Ans :

```
#Length : Gives the length of string
str1="Mohit Sharma"
print("Length : ",len(str1))

#Lower : Convert the string in lower
print("Lower : ",str1.lower())

#Casefold : Convert string strictly to lower including special character
str2 = "Mohit Sharma ß"
print("Casefold : ",str2.casefold())

#upper : Convert the string to upper case
print("Upper : ",str1.upper())

#title : Convert the first Character of all words in sentence to upper case
print("Title : ",str1.title())

#capitalize : Convert the first Character of first word in sentence to upper case
print("Capitalize : ",str1.capitalize())

#Strip : Remove all the leading and trailing spaces
str3 = "   Mohit Sharma    "
print("strip : ",str3.strip())

#Replace : Replace the particular character.
print("Replace : ",str1.replace('h','T',2))

#Find : Gives the index of particular character
print("Find : ",str1.find('h'))

#index : Gives the index of the particular letter
print("Index : ",str1.index('a'))

#Startswith :
print("Startswith M : ",str1.startswith('M'))
print("Startswith N : ",str1.startswith('N'))

#Endswith
print("Endswith a ",str1.endswith('a'))
```

```
print("Endswith P ",str1.endswith('P'))

#Split : Split the string with particular character and gives list
str5="Sun rises from east"
print("Split : ",str5.split("s",2))

#join : Join the 2 strings
print("abc".join("xyz"))

#isalpha
print("abc".isalpha())

#isdigit
print("123sd".isdigit())

#isalnum
print("abc123".isalnum())
print("abc".isalnum())
print("abc123#".isalnum())

#zfill : Fill the string with 0
print("abc".zfill(10))

#center :
print("abc".center(15,"*"))
print("abc".center(16,"*"))
```

Q : 17 Understanding the role of break, continue, and pass in Python loops.

Ans :

1. The break Statement

The break statement is used to exit the loop immediately, regardless of the loop condition.

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

2. The continue Statement

The continue statement is used to skip the current iteration and move to the next iteration of the loop.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

3. The pass Statement

The pass statement does nothing. It is used as a placeholder where a statement is required syntactically but no action is needed.

```
for i in range(1, 4):
    pass
```

Q : 18 String slicing.

Ans :

```
str1 = "Sun rises from east"
print("1 : ",str1[2:])
print("2 : ",str1[2:7])
print("3 : ",str1[2:15:2])
print("4 : ",str1[:8])
print("5 : ",str1[-4:-1])
print("6 : ",str1[::-1])
```